

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF VÀ VECTOR DATABASE

Tổng quan và các phương pháp xử lý văn bản tiên tiến với NLP

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

https://cybersoft.edu.vn/



Nội dung & kết quả

- Các bài toán buổi 2
- Quy trình thực hiện
- Vector Database
- Term Frequency (TF)
- Inverse Document Frequency (IDF)
- Machine Learning cho NLP

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Ứng dụng của TF-IDF



Chắc hẳn các bạn cũng biết phần mềm này chứ

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Ứng dụng của TF-IDF



turnitin

Đây là phần mềm được sử dụng khá phổ biến trong các trường đại học được sử dụng để **kiểm tra đạo văn**

CYBERSOFT

Ứng dụng của TF-IDF

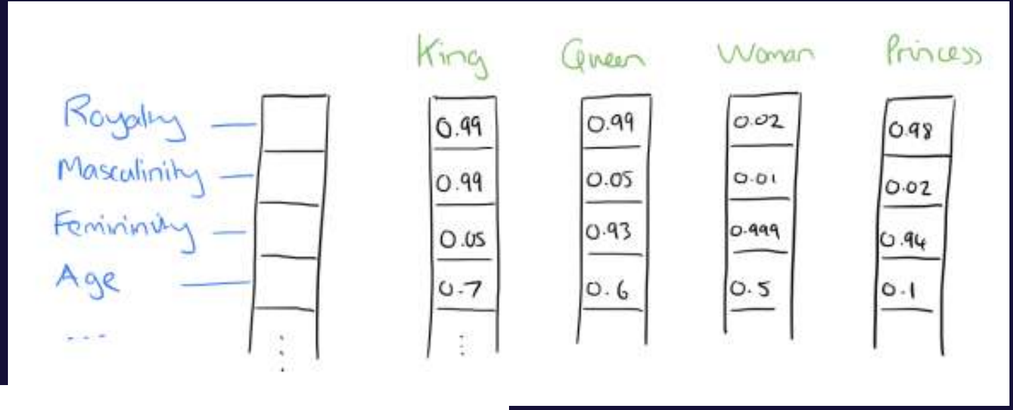


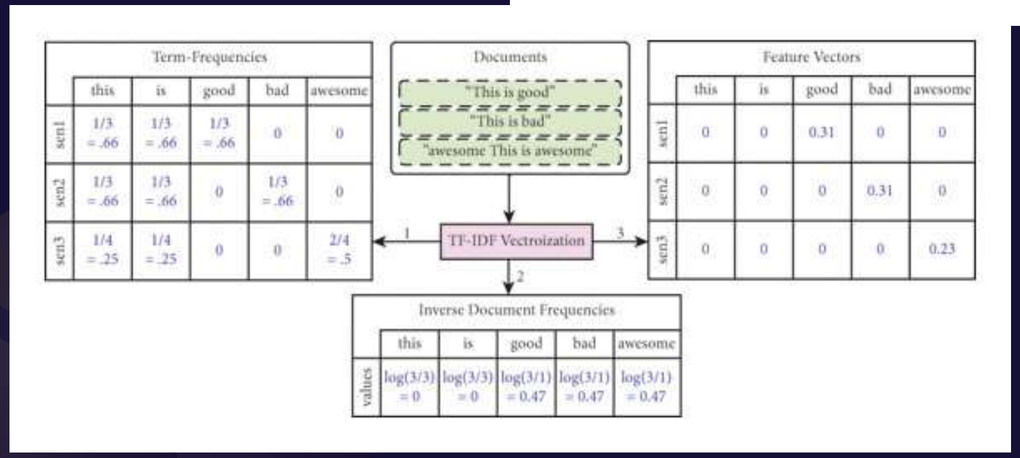
Hôm nay chúng ta cùng tìm hiểu về cơ chế check đạo văn này thông qua tìm hiểu **TF-IDF** và **vector database** nhé

Kiến thức học được

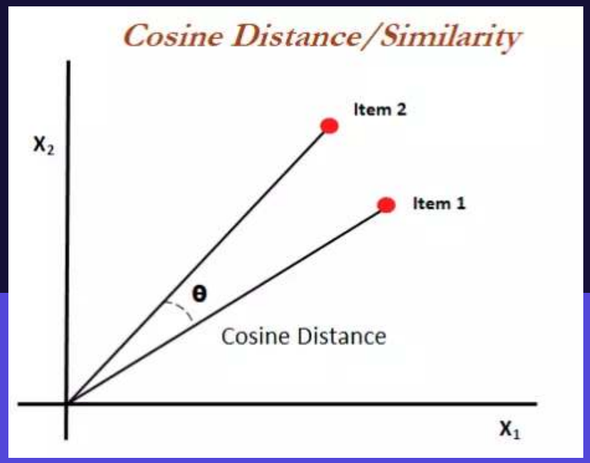
Mục tiêu







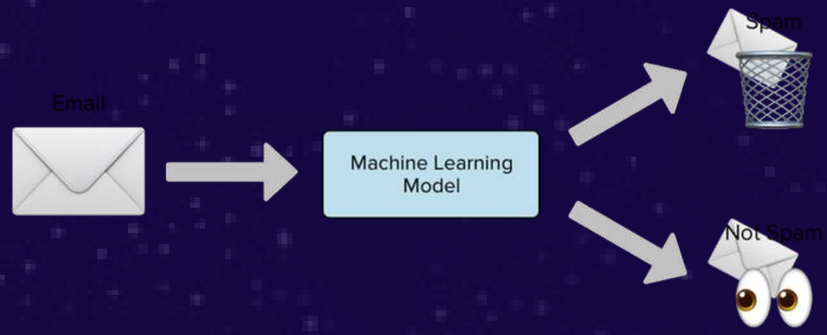
Ứng dụng **TF-IDF** trong việc vector hoá văn bản



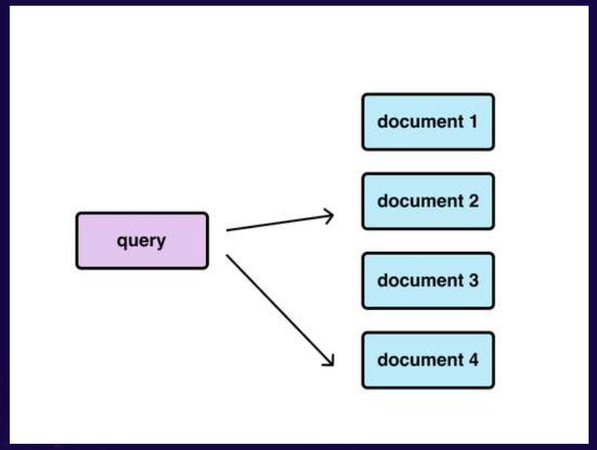
Đo độ tương đồng ứng dụng **cosine similarity**

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

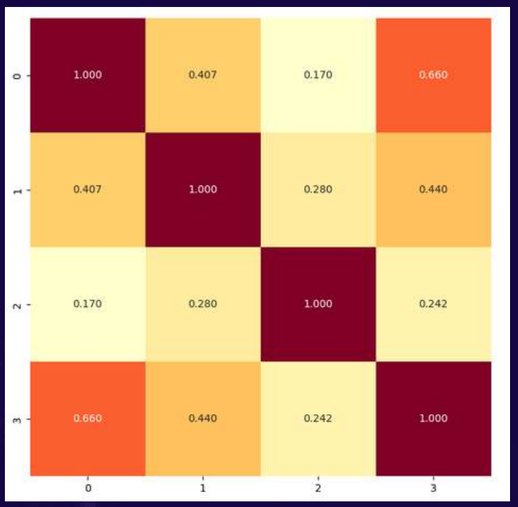
Các bài toán



Text Classification sử dụng Vector DB

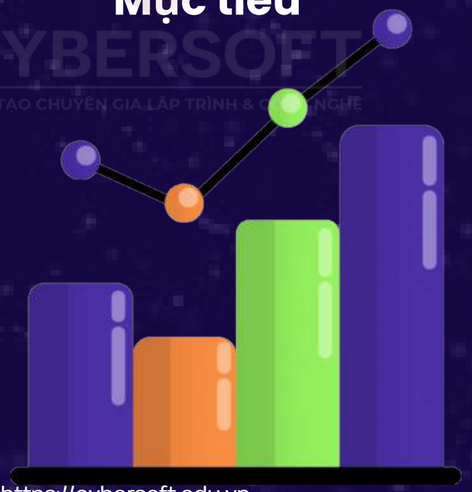


Text Retrieval



Plagiarism Check

Mục tiêu

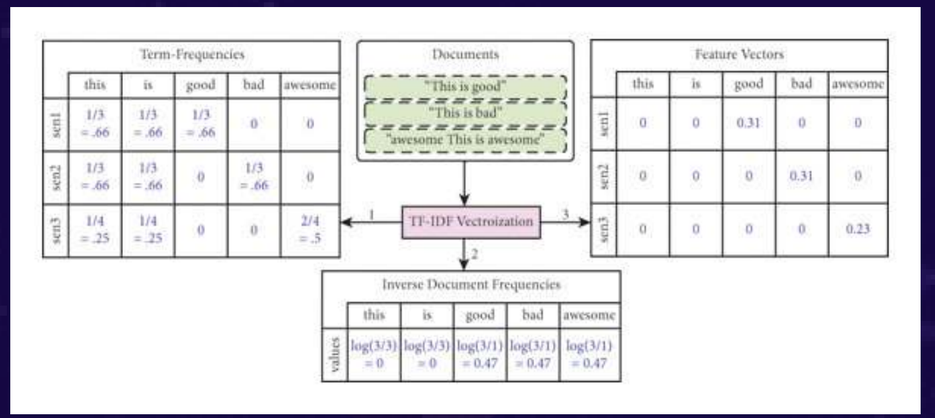


CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Quy trình tổng thể các dự án

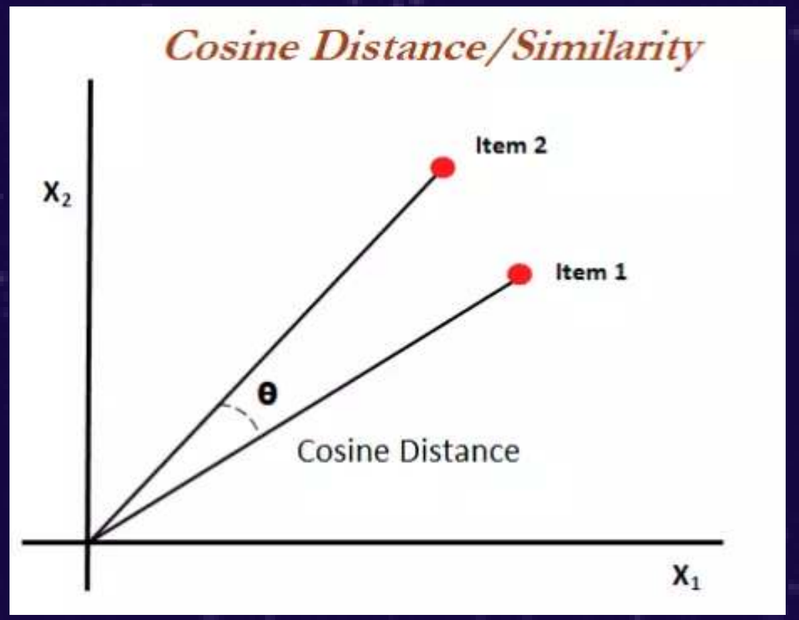


Tokenization



Ứng dụng **TF-IDF** để build **vector database**

Cosine Similarity để tìm kiếm kết quả có độ tương đồng cao



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Chắc hẳn các bạn còn nhớ

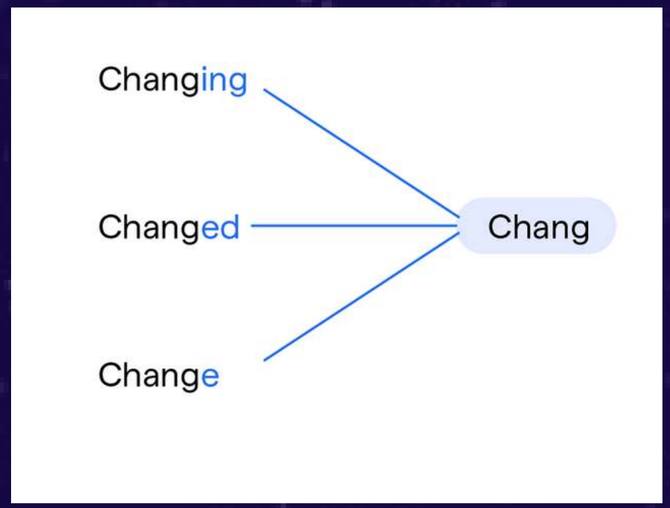
Tokenization

Chia văn bản thô thành các đơn vị nhỏ hơn, gọi là token.



WE LOVE NLP!

[0.17, 0.35, 0.28]



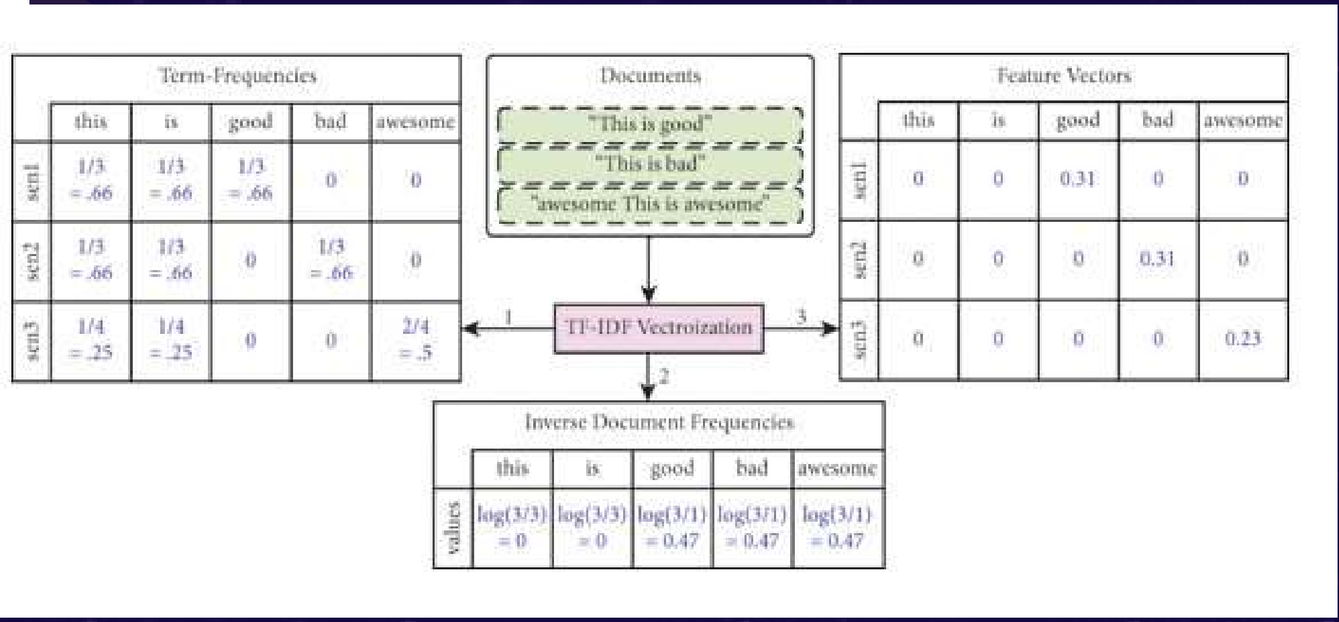
Token có thể là từ, cụm từ, câu, hoặc thậm chí ký tự

BERSOFT
CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Vậy chúng ta cùng tìm hiểu các từ được vector hoá như thế nào nhé?



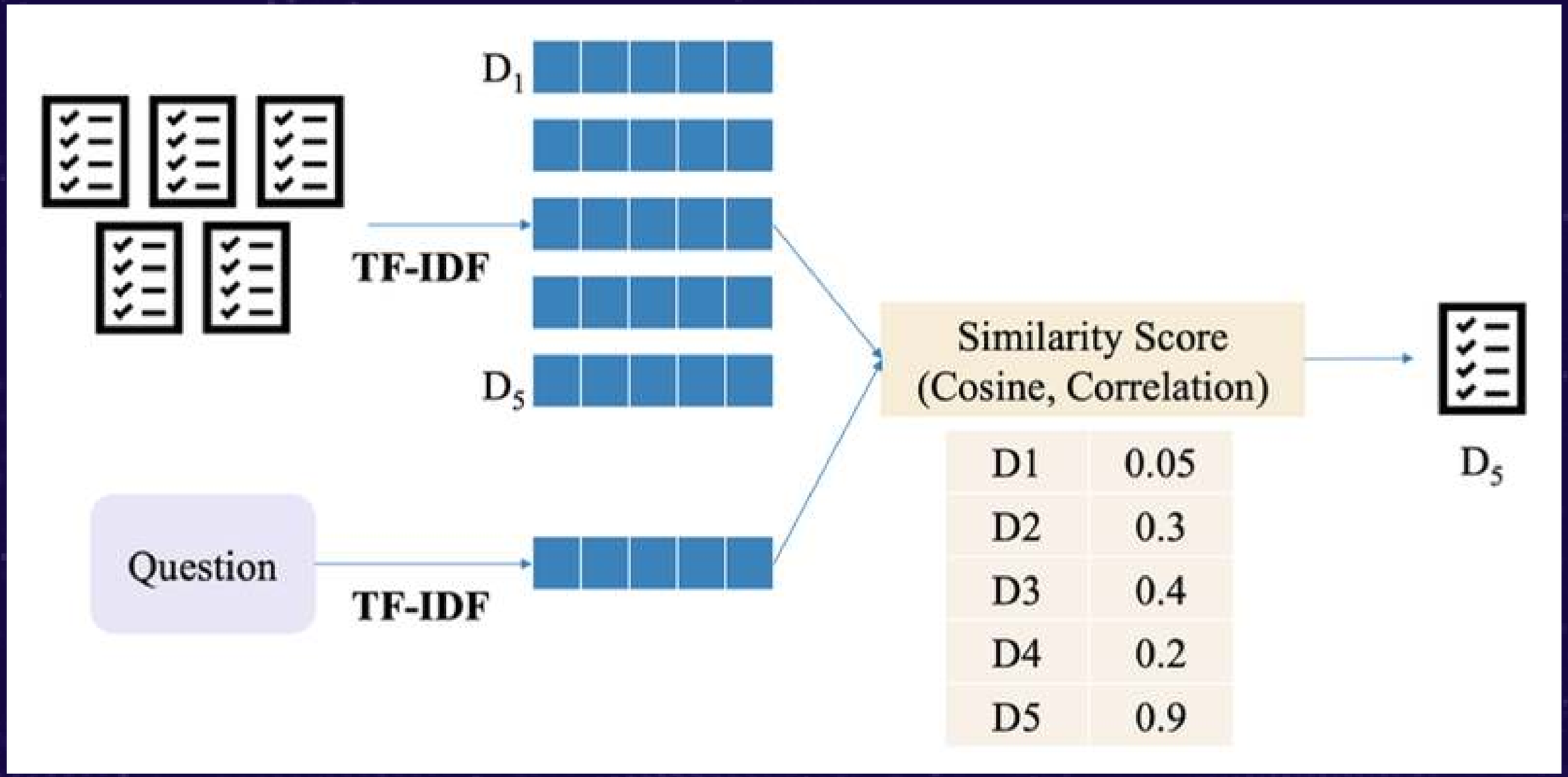
Text Vectorization



TF-IDF & Vector Database

Term Frequency - Inverse Document Frequency (TF-IDF)

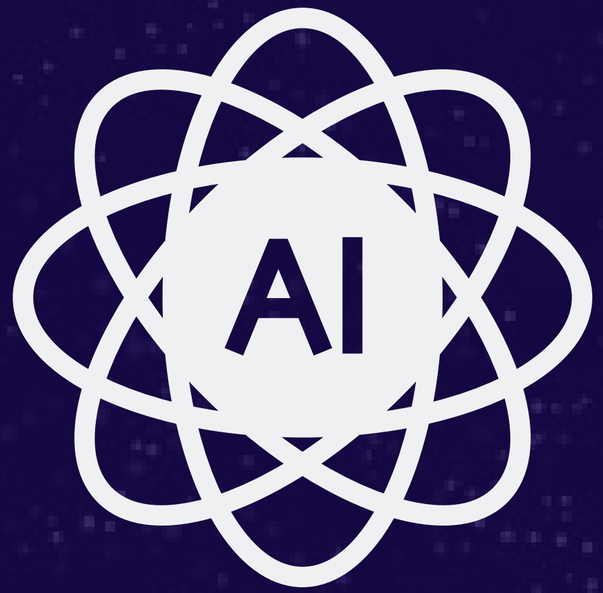
TF-IDF là một kỹ thuật thống kê để đánh giá tầm quan trọng của một từ (term) trong một văn bản (document) so với toàn bộ bộ sưu tập văn bản (corpus).



Term Frequency (TF) là gì?

TF là viết tắt của **Term Frequency** (Tần suất Từ), biểu thị tỷ lệ số lần một từ cụ thể xuất hiện trong văn bản so với tổng số từ trong văn bản.

"AI là tương lai của công nghệ. AI sẽ thay đổi công nghệ trong tương lai."

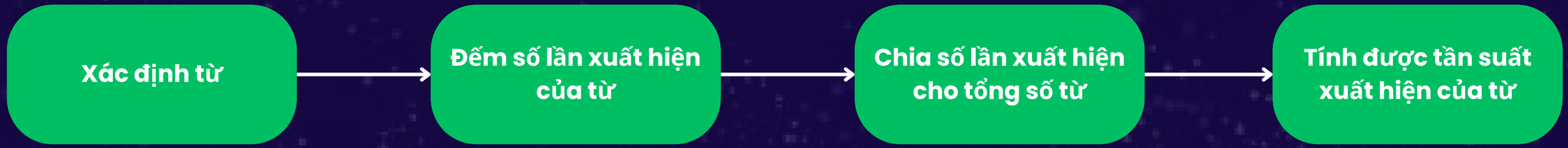


ĐƯỢC ĐỀ CẬP BAO NHIÊU LẦN?

TẦN SUẤT XUẤT HIỆN CỦA “AI"
TRONG CÂU

Term Frequency (TF) là gì? - các bước tính được TF

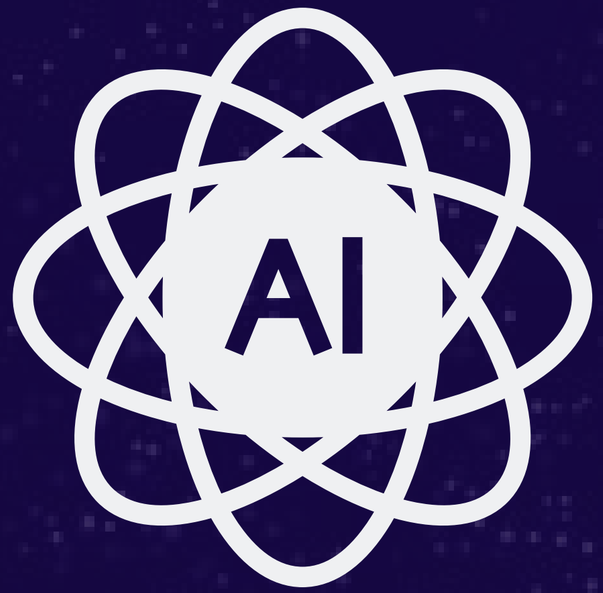
"AI là tương lai của công nghệ. AI sẽ thay đổi công nghệ trong tương lai."



Term Frequency (TF) là gì? - Xác định từ

TF là viết tắt của **Term Frequency** (Tần suất Từ), biểu thị tỷ lệ số lần một từ cụ thể xuất hiện trong văn bản so với tổng số từ trong văn bản.

"AI là tương lai của công nghệ. AI sẽ thay đổi công nghệ trong tương lai."

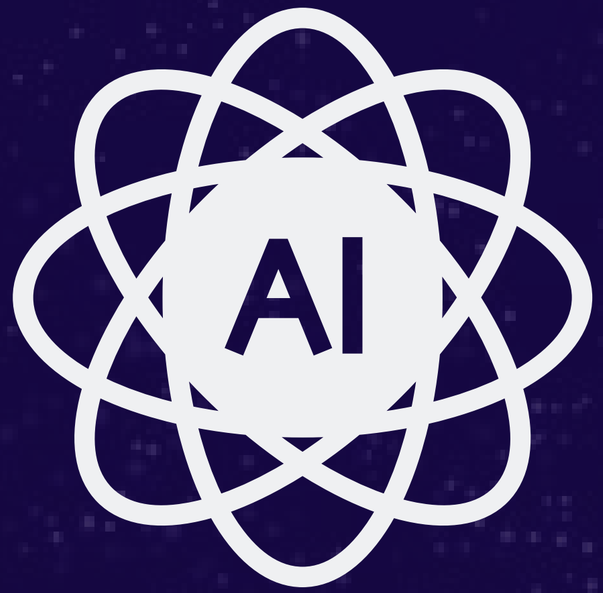


$$AI$$

Term Frequency (TF) là gì? - Đếm số lần xuất hiện của từ

TF là viết tắt của **Term Frequency** (Tần suất Từ), biểu thị tỷ lệ số lần một từ cụ thể xuất hiện trong văn bản so với tổng số từ trong văn bản.

"AI là tương lai của công nghệ. AI sẽ thay đổi công nghệ trong tương lai."



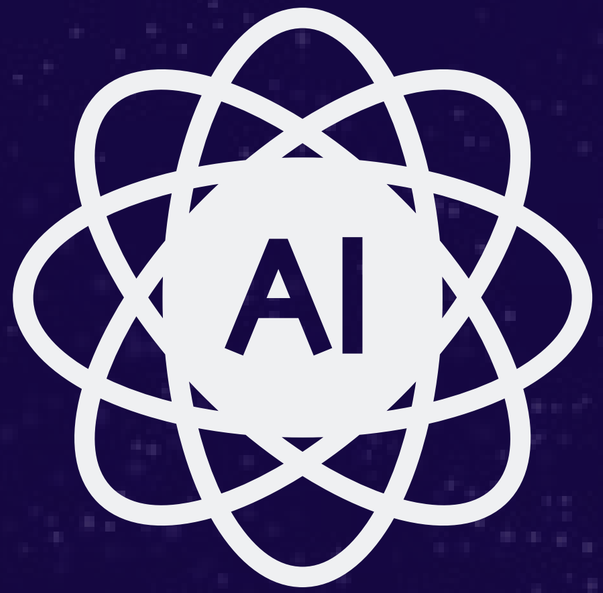
ĐƯỢC ĐỀ CẬP BAO NHIÊU LẦN?

2 LẦN

Term Frequency (TF) là gì? - Chia số lần xuất hiện cho tổng thể

TF là viết tắt của **Term Frequency** (Tần suất Từ), biểu thị tỷ lệ số lần một từ cụ thể xuất hiện trong văn bản so với tổng số từ trong văn bản.

"AI là tương lai của công nghệ. AI sẽ thay đổi công nghệ trong tương lai."



ĐƯỢC ĐỀ CẬP BAO NHIÊU LẦN?

2 LẦN

TẦN SUẤT XUẤT HIỆN CỦA “AI" TRONG CÂU

2 LẦN ĐƯỢC ĐỀ CẬP TRONG TỔNG SỐ 16 TỪ

2/16 = ⅛ = 0.125

Term Frequency (TF) là gì? - Công thức tính tần suất

TF là viết tắt của **Term Frequency** (Tần suất Từ), biểu thị tỷ lệ số lần một từ cụ thể xuất hiện trong văn bản so với tổng số từ trong văn bản.

$$tf_{t,d} = \frac{count(t, d)}{len(d)}$$

count(t, d): số lần xuất hiện của từ **t** trong **document d**
len(d): tổng số từ trong **document d**

Inverse Document Frequency (IDF) là gì?

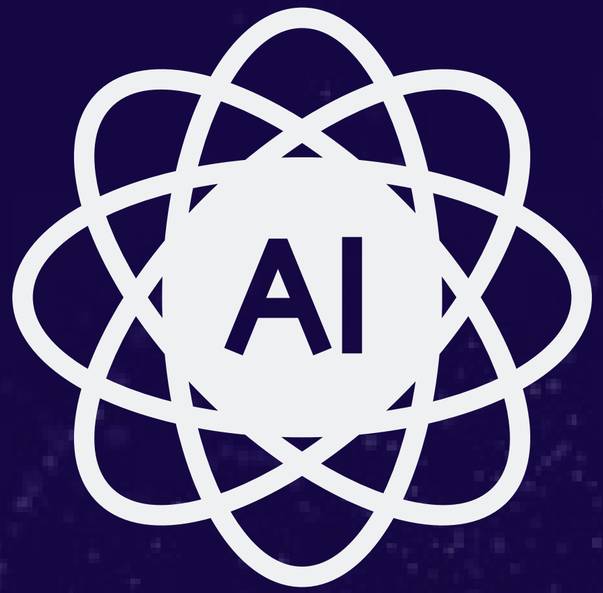
Dùng để đo lường độ hiếm của một từ trong toàn bộ **corpus** (bộ sưu tập văn bản). **IDF** nhấn mạnh rằng các từ **xuất hiện ít** trong corpus (**hiếm gặp**) thường mang ý nghĩa quan trọng hơn các từ phổ biến (như stop-words).

"AI là tương lai của công nghệ."

"Công nghệ thay đổi thế giới."

"AI và công nghệ phát triển nhanh."

"Thế giới cần sự sáng tạo."



ĐƯỢC ĐỀ CẬP TRONG BAO NHIÊU VĂN BẢN?

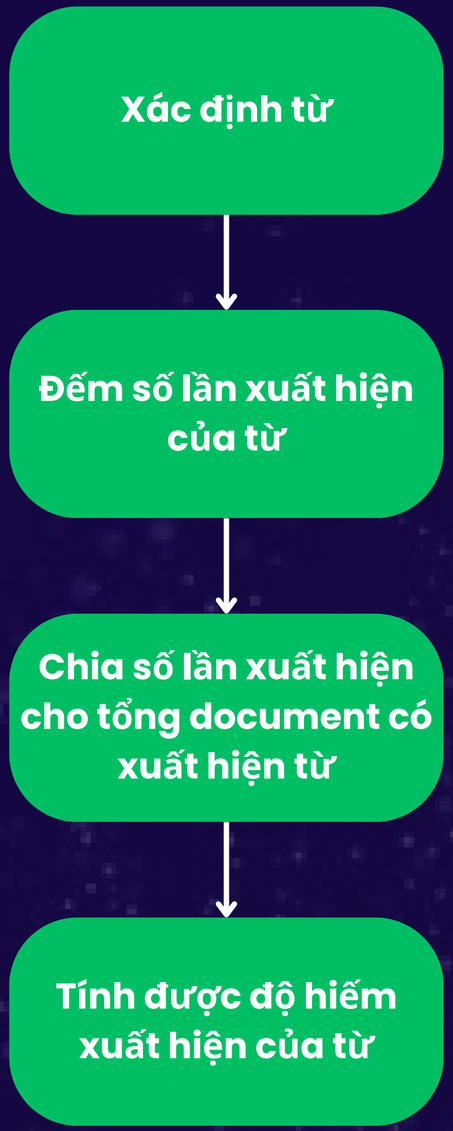
Inverse Document Frequency (IDF) là gì?

"AI là tương lai của công nghệ."

"Công nghệ thay đổi thế giới."

"AI và công nghệ phát triển nhanh."

"Thế giới cần sự sáng tạo."



Inverse Document Frequency (IDF) là gì?

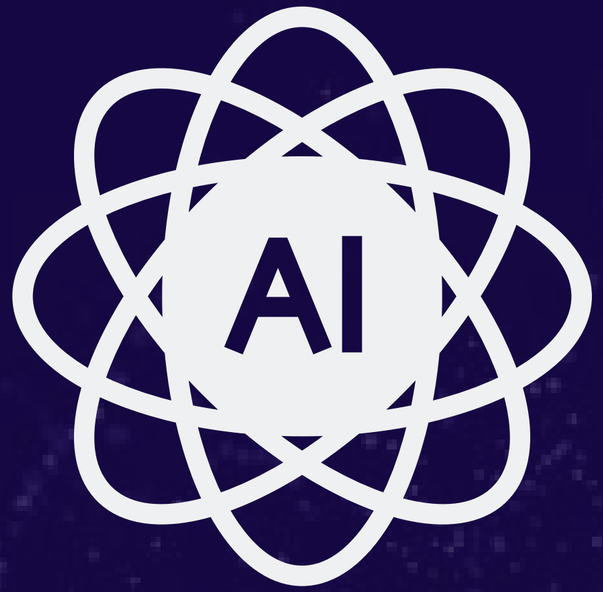
Dùng để đo lường độ hiếm của một từ trong toàn bộ **corpus** (bộ sưu tập văn bản). **IDF** nhấn mạnh rằng các từ **xuất hiện ít** trong corpus (**hiếm gặp**) thường mang ý nghĩa quan trọng hơn các từ phổ biến (như stop-words).

"**AI** là tương lai của công nghệ."

"Công nghệ thay đổi thế giới."

"**AI** và công nghệ phát triển nhanh."

"Thế giới cần sự sáng tạo."



ĐƯỢC ĐỀ CẬP TRONG BAO NHIÊU VĂN BẢN?

XUẤT HIỆN TRONG 2 VĂN BẢN

Inverse Document Frequency (IDF) là gì?

Dùng để đo lường độ hiếm của một từ trong toàn bộ **corpus** (bộ sưu tập văn bản). **IDF** nhấn mạnh rằng các từ **xuất hiện ít** trong corpus (**hiếm gặp**) thường mang ý nghĩa quan trọng hơn các từ phổ biến (như stop-words).

"AI là tương lai của **công nghệ**."

"**Công nghệ** thay đổi thế giới."

"AI và **công nghệ** phát triển nhanh."

"Thế giới cần sự sáng tạo."



CÔNG NGHỆ

ĐƯỢC ĐỀ CẬP TRONG BAO NHIÊU VĂN BẢN?

XUẤT HIỆN TRONG 3 VĂN BẢN

Inverse Document Frequency (IDF) là gì?

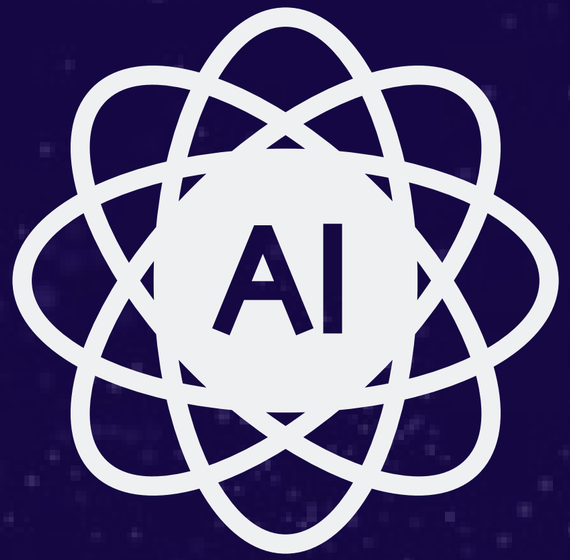
Nếu xét về tần suất xuất hiện



CÔNG NGHỆ

XUẤT HIỆN TRONG
3 VĂN BẢN





XUẤT HIỆN TRONG
2 VĂN BẢN

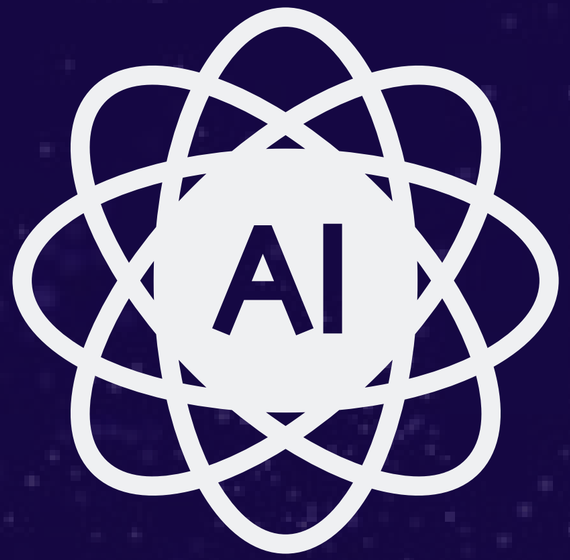
Inverse Document Frequency (IDF) là gì?

NHƯNG nếu xét về độ hiếm của từ trong **corpus**



CÔNG NGHỆ





XUẤT HIỆN TRONG
3 VĂN BẢN

XUẤT HIỆN TRONG
2 VĂN BẢN

Inverse Document Frequency (IDF) là gì?

Vậy **“ĐỘ HIẾM"** của từ được tính như thế nào?

$$idf_t = log \frac{N+1}{df_t+1} + 1$$

N: số **document d** trong **corpus** (bộ sưu tập văn bản)
dft: tổng số **document d** có xuất hiện từ **t**

Inverse Document Frequency (IDF) là gì?

```
v = TfidfVectorizer()
v.fit(corpus)
transform_output = v.transform(corpus)
```

```
#let's print the idf of each word:

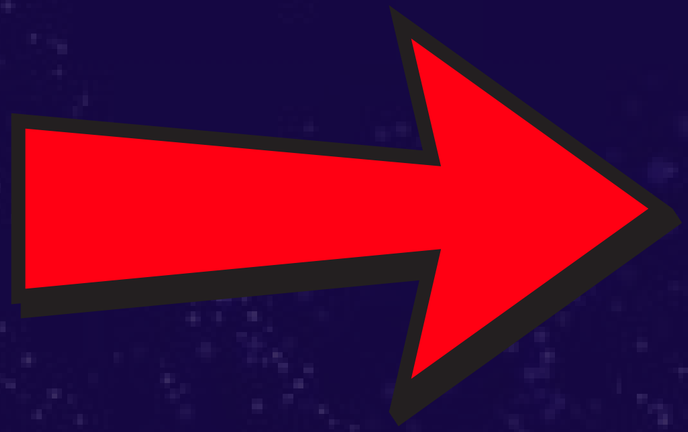
all_feature_names = v.get_feature_names_out()

for word in all_feature_names:

    #let's get the index in the vocabulary
    indx = v.vocabulary_.get(word)

    #get the score
    idf_score = v.idf_[indx]

    print(f"{word} : {idf_score}")
```



```
already : 2.386294361119891
am : 2.386294361119891
amazon : 2.386294361119891
and : 2.386294361119891
announcing : 1.2876820724517808
apple : 2.386294361119891
are : 2.386294361119891
ate : 2.386294361119891
biryani : 2.386294361119891
dot : 2.386294361119891
eating : 1.9808292530117262
eco : 2.386294361119891
google : 2.386294361119891
grapes : 2.386294361119891
iphone : 2.386294361119891
ironman : 2.386294361119891
is : 1.1335313926245225
loki : 2.386294361119891
microsoft : 2.386294361119891
model : 2.386294361119891
new : 1.2876820724517808
pixel : 2.386294361119891
pizza : 2.386294361119891
surface : 2.386294361119891
tesla : 2.386294361119891
thor : 2.386294361119891
tomorrow : 1.2876820724517808
you : 2.386294361119891
```

Công thức tính TF-IDF

Chúng ta vừa đi qua **TF** và **IDF,** cả hai khái niệm được ứng dụng để hiển thị **mức độ quan trọng** của từ trong văn bản





Công thức tính TF-IDF

Chúng ta vừa đi qua **TF** và **IDF,** cả hai khái niệm được ứng dụng để hiển thị **mức độ quan trọng** của từ trong văn bản

$$w_{t,d} = tf_{t,d} \times idf_t$$

- Giá trị trọng số **wt,d** cho từ **t** trong văn bản **d**:
- **IDF** làm **giảm trọng số** của các từ **rất phổ biến** trên toàn **corpus** và **tăng trọng số** các **từ hiếm**.
- **TF-IDF** cho một văn bản chỉ đơn giản là **trọng số** cho mỗi từ trong văn bản đó. Hiển thị mức độ quan trọng của từ trong **corpus**

Bài tập tính TF-IDF

[“dog", “bites", “man"]
[“man", “bites", “dog"]
[“dog", “eat", “meat"]
[“man", “eat", “food"]

Tạo bảng tính TF (Term Frequency)

Tạo bảng tính IDF (Inverse Document Frequency)

Tạo bảng tính TF-IDF

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập tính TF-IDF

["dog", "bites", "man"]

["man", "bites", "dog"]

["dog", "eat", "meat"]

["man", "eat", "food"]

Bảng TF của các từ trong corpus

	bites	dog	eats	food	man	meat
D1	1/3	1/3	0	0	1/3	0
D2	1/3	1/3	0	0	1/3	0
D3	0	1/3	1/3	0	0	1/3
D4	0	0	1/3	1/3	1/3	0

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập tính TF-IDF

["dog", "bites", "man"]

["man", "bites", "dog"]

["dog", "eat", "meat"]

["man", "eat", "food"]

Bảng TF của các từ trong corpus

	bites	dog	eats	food	man	meat
D1	1/3	1/3	0	0	1/3	0
D2	1/3	1/3	0	0	1/3	0
D3	0	1/3	1/3	0	0	1/3
D4	0	0	1/3	1/3	1/3	0

Bảng IDF của các từ trong corpus

bites	dog	eats	food	man	meat
1.511	1.223	1.511	1.916	1.223	1.916

CYBERSOFT
LẬP TRÌNH & CÔNG NGHỆ

Bài tập tính TF-IDF

["dog", "bites", "man"]

["man", "bites", "dog"]

["dog", "eat", "meat"]

["man", "eat", "food"]

Bảng TF của các từ trong corpus

	bites	dog	eats	food	man	meat
D1	1/3	1/3	0	0	1/3	0
D2	1/3	1/3	0	0	1/3	0
D3	0	1/3	1/3	0	0	1/3
D4	0	0	1/3	1/3	1/3	0

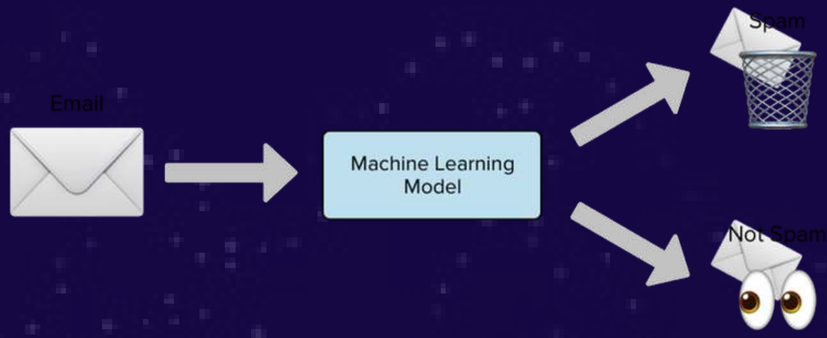
Bảng IDF của các từ trong corpus

bites	dog	eats	food	man	meat
1.511	1.223	1.511	1.916	1.223	1.916

Bảng TF-IDF của các từ trong corpus

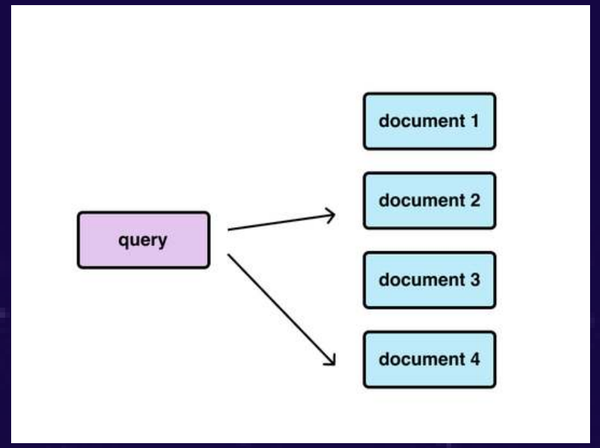
	bites	dog	eats	food	man	meat
D1	0.504	0.408	0	0	0.400	0
D2	0.504	0.408	0	0	0.408	0
D3	0	0.408	0.504	0	0	0.639
D4	0	0	0.504	0.639	0.408	0

Ứng dụng TF-IDF

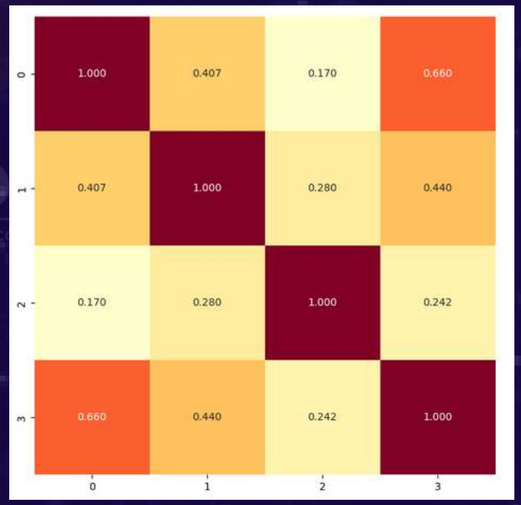


TF-IDF tạo vector đặc trưng cho văn bản, dùng làm input cho mô hình học máy để phân loại.

Sử dụng TF-IDF để embedding văn bản và query thành vector, sau đó tính độ tương đồng (thường là Cosine Similarity) để tìm các văn bản liên quan nhất.

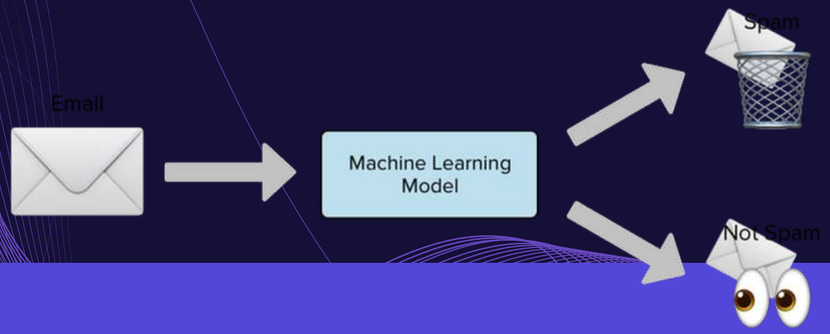


So sánh vector TF-IDF của văn bản mới với corpus để phát hiện độ tương đồng cao, chỉ ra đạo văn.





Bài tập trên lớp



Phân loại Email Spam với Vector Database



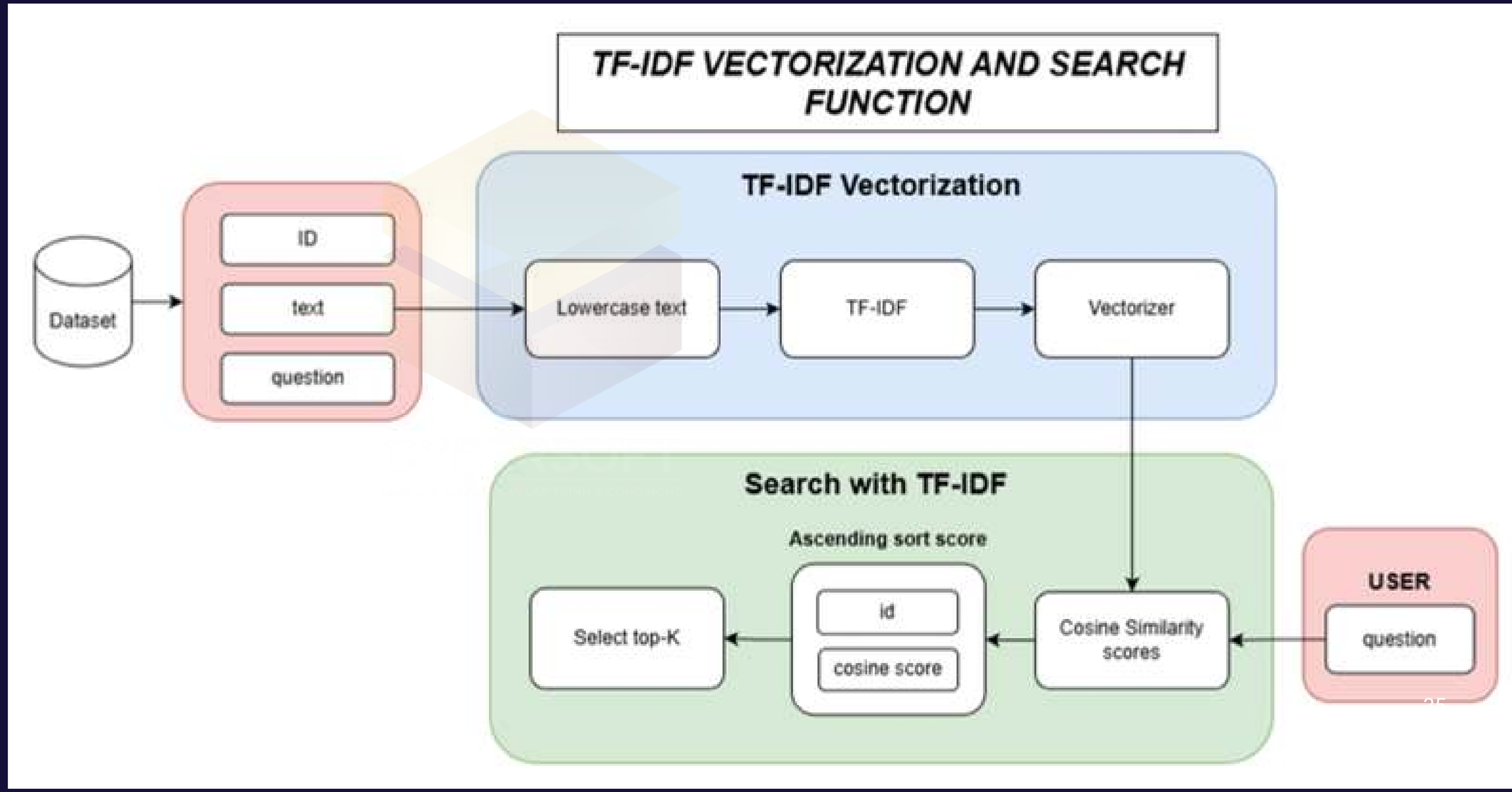
Bài tập thực hành #1

NLP với Vector Embedding

Phân loại Email Spam với Vector Database

Bài tập trên lớp

PIPELINE



Các bước thực hiện

1. Cài đặt thư viện
2. Import thư viện
3. Đọc dữ liệu
4. Phân chia dữ liệu
5. Chuẩn bị mô hình Embedding
6. Vector hoá dữ liệu
7. Xây dựng Vector DB
8. Pipeline phân loại
9. Đánh giá mô hình
10. Lưu mô hình

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH

Cài đặt thư viện

```
!pip install -qq faiss-cpu
!pip install -qq transformers
!pip install -qq pandas
!pip install -qq numpy
!pip install -qq scikit-learn
!pip install -qq tqdm
```

Import thư viện

```
import pandas as pd
import numpy as np
import torch
import torch.nn.functional as F
import faiss
from transformers import AutoTokenizer, AutoModel
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tqdm import tqdm
```

Đọc dữ liệu

```
DATASET_PATH = '/content/cls_spam_text.csv'
df = pd.read_csv(DATASET_PATH)
df
```

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Phân chia dữ liệu

```
messages = df['Message'].values.tolist()
labels = df['Category'].values.tolist()
```



Chuẩn bị mô hình Embedding

```python
# Load embedding model
MODEL_NAME = 'intfloat/multilingual-e5-base'
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModel.from_pretrained(MODEL_NAME)

# Set device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
model.eval()

print(f'Using device: {device}')
print(f'Model loaded: {MODEL_NAME}')
```

CYBERSOFT

Tải và chuẩn bị một mô hình **embedding văn bản** đa ngôn ngữ **(intfloat/multilingual-e5-base)** từ **Hugging Face** để tạo **vector đặc trưng** từ văn bản.

Chuẩn bị mô hình Embedding

Hàm pooling

```python
def average_pool(last_hidden_states, attention_mask):
    last_hidden = last_hidden_states.masked_fill(
        ~attention_mask[..., None].bool(), 0.0
    )
    return last_hidden.sum(dim=1) / attention_mask.sum(dim=1)[..., None]
```

Thực hiện **average pooling** trên **hidden states** của mô hình **Transformer**, sử dụng mask để bỏ qua **padding** (thêm các token giả để làm đều độ dài sequence).

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Chuẩn bị mô hình Embedding

Hàm Encode

```python
def encode_text(batch_texts, model, tokenizer, device):
    batch_dict = tokenizer(
        batch_texts,
        max_length=512,
        padding=True,
        truncation=True,
        return_tensors='pt'
    )

    # Move to device
    batch_dict = {k: v.to(device) for k, v in batch_dict.items()}

    # Generate embeddings
    with torch.no_grad():
        outputs = model(**batch_dict)
        batch_embeddings = average_pool(
            outputs.last_hidden_state, batch_dict['attention_mask'])
        # Normalize embeddings
        batch_embeddings = F.normalize(batch_embeddings, p=2, dim=1)

    return batch_embeddings.cpu().numpy()
```

Hàm này dùng để **tạo embedding văn bản** từ một batch văn bản sử dụng mô hình **Transformer**.

Chuẩn bị mô hình Embedding

Hàm Encode

```python
def encode_text(batch_texts, model, tokenizer, device):
    batch_dict = tokenizer(
        batch_texts,
        max_length=512,
        padding=True,
        truncation=True,
        return_tensors='pt'
    )

    # Move to device
    batch_dict = {k: v.to(device) for k, v in batch_dict.items()}

    # Generate embeddings
    with torch.no_grad():
        outputs = model(**batch_dict)
        batch_embeddings = average_pool(
            outputs.last_hidden_state, batch_dict['attention_mask'])
        # Normalize embeddings
        batch_embeddings = F.normalize(batch_embeddings, p=2, dim=1)

    return batch_embeddings.cpu().numpy()
```

Embedding là vector số biểu diễn ý nghĩa ngữ nghĩa của văn bản, có thể dùng cho các nhiệm vụ như tìm kiếm **(semantic search)**, phân loại, hoặc **clustering**.

Vector hoá dữ liệu

Hàm tạo Sentence Embedding

```python
def get_embeddings(texts, model, tokenizer, device, batch_size=32):
    """Generate embeddings for a list of texts"""
    embeddings = []

    for i in tqdm(range(0,len(texts),batch_size),desc="Generating embeddings"):
        batch_texts = texts[i:i+batch_size]

        # Add passage prefix for better retrieval performance
        batch_texts_with_prefix = [f"passage: {text}" for text in batch_texts]

        batch_embeddings = encode_text(batch_texts, model, tokenizer, device)
        embeddings.append(batch_embeddings)

    return np.vstack(embeddings)
```

Vector hoá dữ liệu

Tạo Sentence Embedding

```
# Prepare labels
le = LabelEncoder()
y = le.fit_transform(labels)
print(f'Classes: {le.classes_}')

# Generate embeddings for all messages
print(f"Generating embeddings for {len(messages)} messages...")
X_embeddings = get_embeddings(messages, model, tokenizer, device)
print(f"Embeddings shape: {X_embeddings.shape}")

# Create metadata for each document
metadata = []
for i, (message, label) in enumerate(zip(messages, labels)):
    metadata.append({
        'index': i,
        'message': message,
        'label': label,
        'label_encoded': y[i]
    })

print(f"Created metadata for {len(metadata)} documents")
```

```
Classes: ['ham' 'spam']
Generating embeddings for 5572 messages...
Generating embeddings: 100%|██████████| 175/175
Created metadata for 5572 documents
```

Xây dựng vector DB

Phân chia dữ liệu thành tập train và test

```
# Split data into train and test (90% train, 10% test)
TEST_SIZE = 0.1
SEED = 42

train_indices, test_indices = train_test_split(
    range(len(messages)),
    test_size=TEST_SIZE,
    stratify=y,
    random_state=SEED
)
```

Xây dựng vector DB

Phân chia dữ liệu thành tập train và test

```
# Split embeddings and metadata
X_train_emb = X_embeddings[train_indices]
X_test_emb = X_embeddings[test_indices]
y_train = y[train_indices]
y_test = y[test_indices]

train_metadata = [metadata[i] for i in train_indices]
test_metadata = [metadata[i] for i in test_indices]

print(f"Train size: {len(X_train_emb)}")
print(f"Test size: {len(X_test_emb)}")
print(f"Train label distribution: {np.bincount(y_train)}")
print(f"Test label distribution: {np.bincount(y_test)}")
```

Xây dựng vector DB

Tạo FAISS index

```
# Create FAISS index
embedding_dim = X_train_emb.shape[1]
index = faiss.IndexFlatIP(embedding_dim)  # Inner product for cosine similarity
index.add(X_train_emb.astype('float32'))

print(f"FAISS index created with {index.ntotal} vectors")
```

```
FAISS index created with 5014 vectors
```

Xây dựng pipeline phân loại

```python
def spam_classifier_pipeline(user_input, k=3):
    """
    Complete pipeline for spam classification

    Args:
        user_input (str): Text to classify
        k (int): Number of nearest neighbors to consider

    Returns:
        dict: Classification results with details
    """

    print()
    print(f"***Classifying: '{user_input}'")
    print()
    print(f"***Using top-{k} nearest neighbors")
    print()

    # Get prediction and neighbors
    prediction, neighbors = classify_with_knn(
        user_input, model, tokenizer, device, index, train_metadata, k=k
    )

    # Display results
    print(f"***Prediction: {prediction.upper()}")
    print()

    print("***Top neighbors:")
    for i, neighbor in enumerate(neighbors, 1):
        print(f"{i}. Label: {neighbor['label']} | Score: {neighbor['score']:.4f}")
        print(f"   Message: {neighbor['message']}")
        print()

    # Count label distribution
    labels = [n['label'] for n in neighbors]
    label_counts = {label: labels.count(label) for label in set(labels)}

    return {
        'prediction': prediction,
        'neighbors': neighbors,
        'label_distribution': label_counts
    }
```

CYBERS
ĐÀO TẠO CHUYÊN GIA LẬP TRÌ

Đánh giá mô hình

Test pipeline qua các ví dụ khác nhau

```
# Test với các ví dụ khác nhau
test_examples = [
    "I am actually thinking a way of doing something useful",
    "FREE!! Click here to win $1000 NOW! Limited time offer!",
    # "Hey, can you pick me up at 5pm today?",
    # "URGENT: Your account will be suspended unless you verify your details NOW",
    # "Thanks for the meeting today, let's schedule the next one for next week",
    # "Congratulations! You've won a prize! Call this number to claim it"
]

print("Testing pipeline with different examples:")
print()

for i, example in enumerate(test_examples, 1):
    print(f"\n***Example {i}:")
    result = spam_classifier_pipeline(example, k=3)
    print()
```

Đánh giá mô hình

Test pipeline qua các ví dụ khác nhau

```
Testing pipeline with different examples:


***Example 1:

***Classifying: 'I am actually thinking a way of doing something useful'

***Using top-3 nearest neighbors

***Prediction: HAM

***Top neighbors:
1. Label: ham | Score: 0.8378
   Message: K, I'll work something out

2. Label: ham | Score: 0.8376
   Message: I have gone into get info bt dont know what to do

3. Label: ham | Score: 0.8327
   Message: Same. Wana plan a trip sometme then
```

Đánh giá mô hình

Test pipeline qua các ví dụ khác nhau

```
***Example 2:

***Classifying: 'FREE!! Click here to win $1000 NOW! Limited time offer!'

***Using top-3 nearest neighbors

***Prediction: SPAM

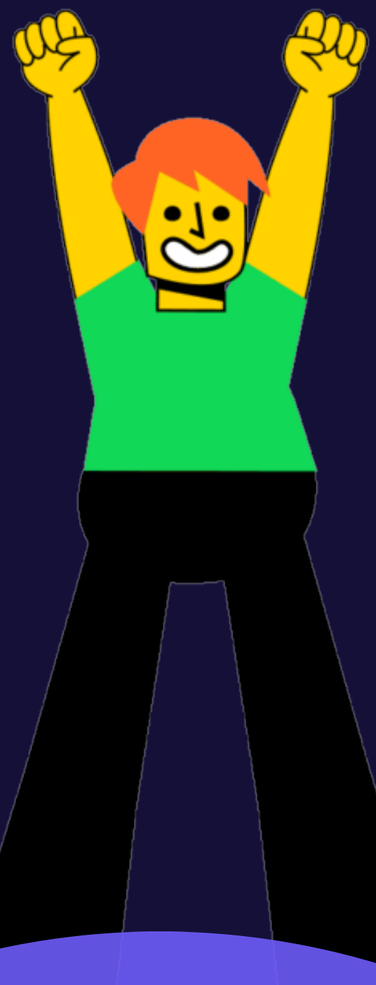
***Top neighbors:
1. Label: spam | Score: 0.8572
   Message: URGENT! You have won a 1 week FREE membership in our £100,000 Prize Jackpot! Txt the word: CLAIM to ...

2. Label: spam | Score: 0.8556
   Message: Win a £1000 cash prize or a prize worth £5000

3. Label: spam | Score: 0.8545
   Message: For your chance to WIN a FREE Bluetooth Headset then simply reply back with "ADP"
```

Vậy là chúng ta vừa hoàn tất xong quy trình vector hoá văn bản ứng dụng TF-IDF

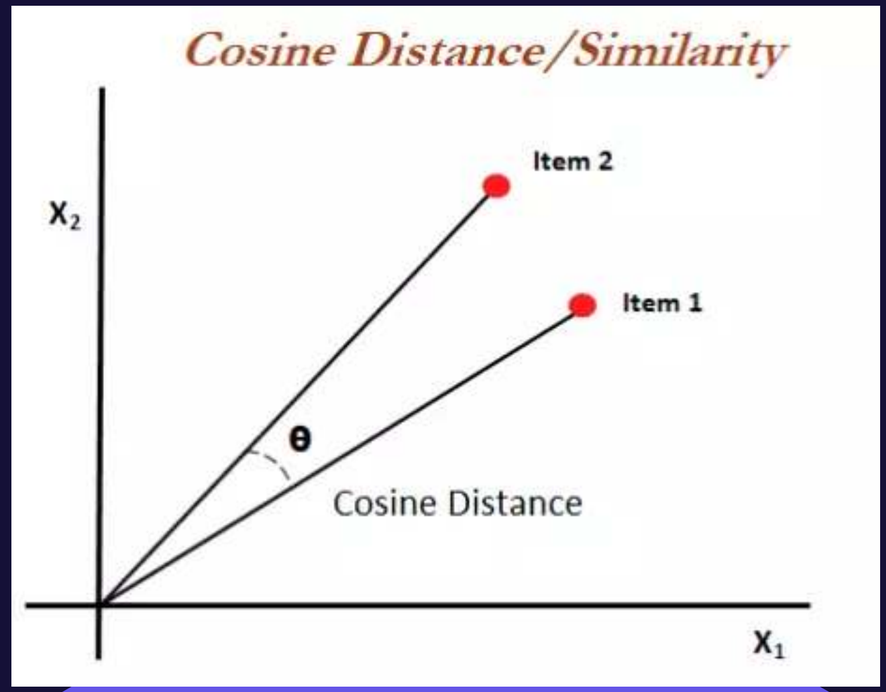
ĐƯỢC ĐỀ CẬP TRONG BAO NHIÊU VĂN BẢN?



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Vậy là chúng ta vừa hoàn tất xong quy trình vector hoá văn bản ứng dụng TF-IDF

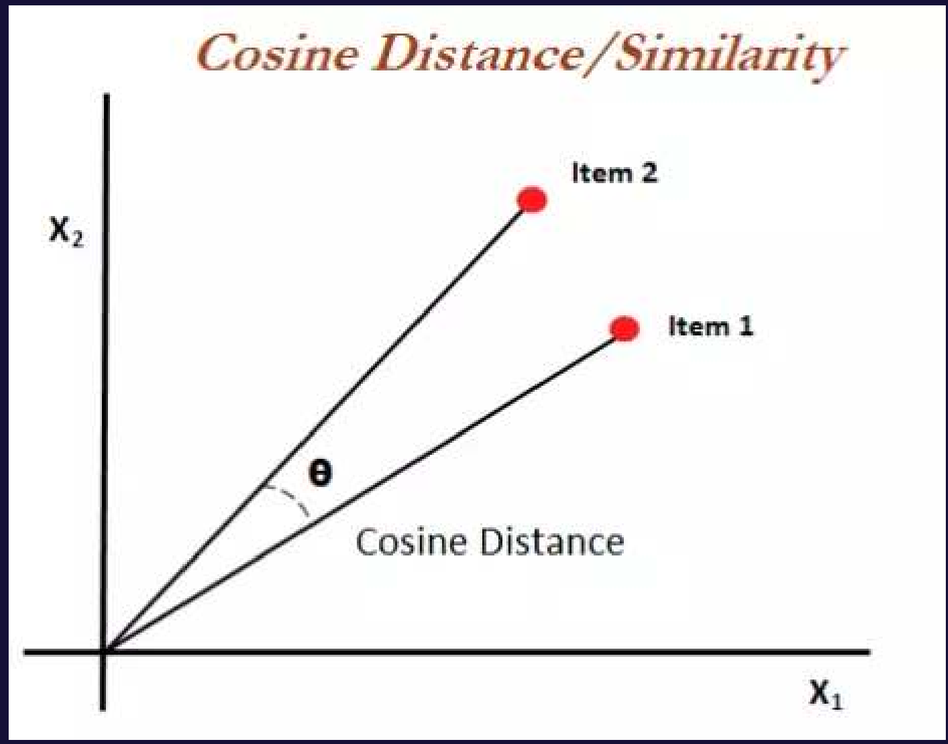
CHÚNG TA CÙNG TÌM HIỂU XEM CÁC VECTOR HOẠT ĐỘNG NHƯ THẾ NÀO TRONG DỰ ÁN KẾ TIẾP NHÉ



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Cosine Similarity

Bài toán Text Retrieval



"Một trận đấu bóng đá kéo dài trong bao lâu"

- "Một trận đấu bóng đá thông thường có hai hiệp , mỗi hiệp 45 phút với khoảng thời gian 15 phút nghỉ giữa hai hiệp ."
- "Luật bóng đá là hệ thống quy định của FIFA trong việc chơi bóng đá , kể cả bóng đá trên sân cỏ chính thức , bóng đá trong nhà,..."
- "Tây Ban Nha cũng là đội vô địch World Cup đầu tiên giữ được sạch lưới tại vòng đấu loại trực tiếp ."

CYBERSOFT

Cosine Similarity

Bài toán Text Retrieval

CÂU TRẢ LỜI CHÍNH XÁC

"Một trận đấu bóng đá kéo dài trong bao lâu"

- "Một trận đấu bóng đá thông thường có hai hiệp , mỗi hiệp 45 phút với khoảng thời gian 15 phút nghỉ giữa hai hiệp ."
- "Luật bóng đá là hệ thống quy định của FIFA trong việc chơi bóng đá , kể cả bóng đá trên sân cỏ chính thức , bóng đá trong nhà,..."
- "Tây Ban Nha cũng là đội vô địch World Cup đầu tiên giữ được sạch lưới tại vòng đấu loại trực tiếp ."



CYBERSOFT

Cosine Similarity

Bài toán Text Retrieval

CÂU TRẢ LỜI CHÍNH XÁC

Chúng ta đã biết cách vector hoá văn bản text rồi

Vậy ta có thể tận dụng các tính chất liên quan đến vector để giải quyết bài này không?



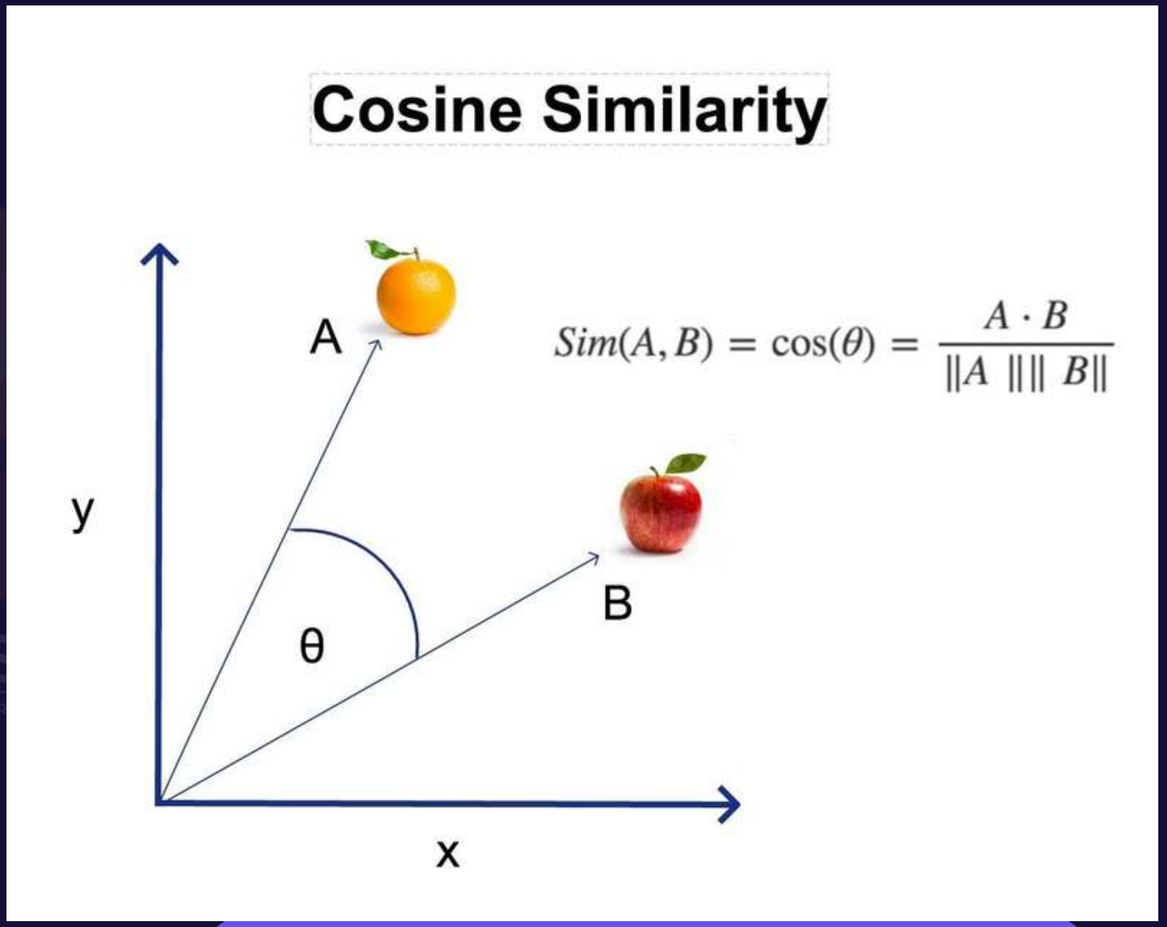
CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Cosine Similarity

Bài toán Text Retrieval

CÁC BẠN CÒN NHỚ COSINE SIMILARITY CHỨ?

Cosine Similarity đo lường mức độ tương đồng giữa hai vector bằng cách tính cosine của góc giữa chúng trong không gian vector. Giá trị này nằm trong khoảng từ **-1 đến 1**:



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH

Kiến trúc hệ thống Text Retrieval



Cosine Similarity

Bài toán Text Retrieval

[0.45, 0.3, 0.12, 0.32,....., 0.77, 0.1, 0.05, 0.2]

→ [0.48, 0.28, 0.15, 0.35,....., 0.80, 0.08, 0.06, 0.22]

→ [0.50, 0.32, 0.10, 0.30,....., 0.75, 0.12, 0.04, 0.18]

→ [0.1, 0.8, 0.9, 0.2,....., 0.05, 0.7, 0.85, 0.6]

CÂU TRẢ LỜI CHÍNH XÁC



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Cosine Similarity

Bài toán Text Retrieval

[0.45, 0.3, 0.12, 0.56,.....,0.77, 0.1, 0.05, 0.2]

→ [0.48, 0.28, 0.15, 0.35,.....,0.80, 0.08, 0.06, 0.22]

COSINE: 0.999

→ [0.50, 0.32, 0.10, 0.30,.....,0.75, 0.12, 0.04, 0.18]

COSINE: 0.998

→ [0.1, 0.8, 0.9, 0.2,.....,0.05, 0.7, 0.85, 0.6]

COSINE: 0.431

CÂU TRẢ LỜI CHÍNH XÁC



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Cosine Similarity

Bài toán Text Retrieval

[0.45, 0.3, 0.12, 0.32,....., 0.77, 0.1, 0.05, 0.2]

```python
def corr_search(question, tfidf_vectorizer, top_d=5):
    # Lowercasing before encoding
    query_embedded = tfidf_vectorizer.transform([question.lower()]).toarray()[0]
    cosine_scores = cosine_similarity([query_embedded], tfidf_vectorizer.transform(context).toarray())[0]

    # Get top k cosine score and index its
    results = []
    for idx in cosine_scores.argsort()[-top_d:][::-1]:
        doc_score = {
            'id': idx,
            'corr_score': cosine_scores[idx]
        }
        results.append(doc_score)
    return results
```

CÂU TRẢ LỜI CHÍNH XÁC



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Cosine Similarity

Bài toán Text Retrieval

CÂU TRẢ LỜI CHÍNH XÁC

"Một trận đấu bóng đá kéo dài trong bao lâu"

- "Một trận đấu bóng đá thông thường có hai hiệp , mỗi hiệp 45 phút với khoảng thời gian 15 phút nghỉ giữa hai hiệp ."

  COSINE: 0.999

- "Luật bóng đá là hệ thống quy định của FIFA trong việc chơi bóng đá , kể cả bóng đá trên sân cỏ chính thức , bóng đá trong nhà,..."

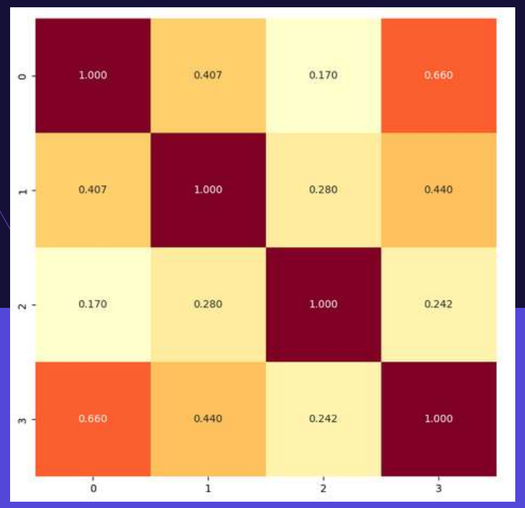
  COSINE: 0.998



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



Bài tập trên lớp



TF-IDF - Plagiarism Checker

CYBERSOFT

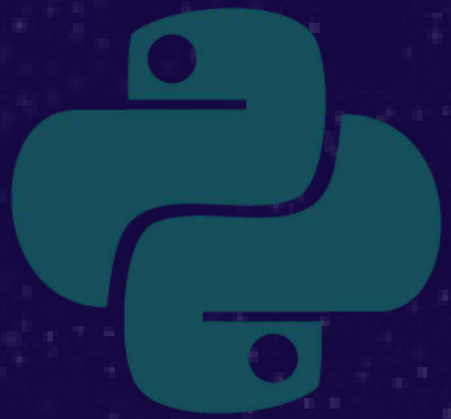
Bài tập trên lớp

CÔNG NGHỆ & FRAMEWORK



NumPy

spaCy

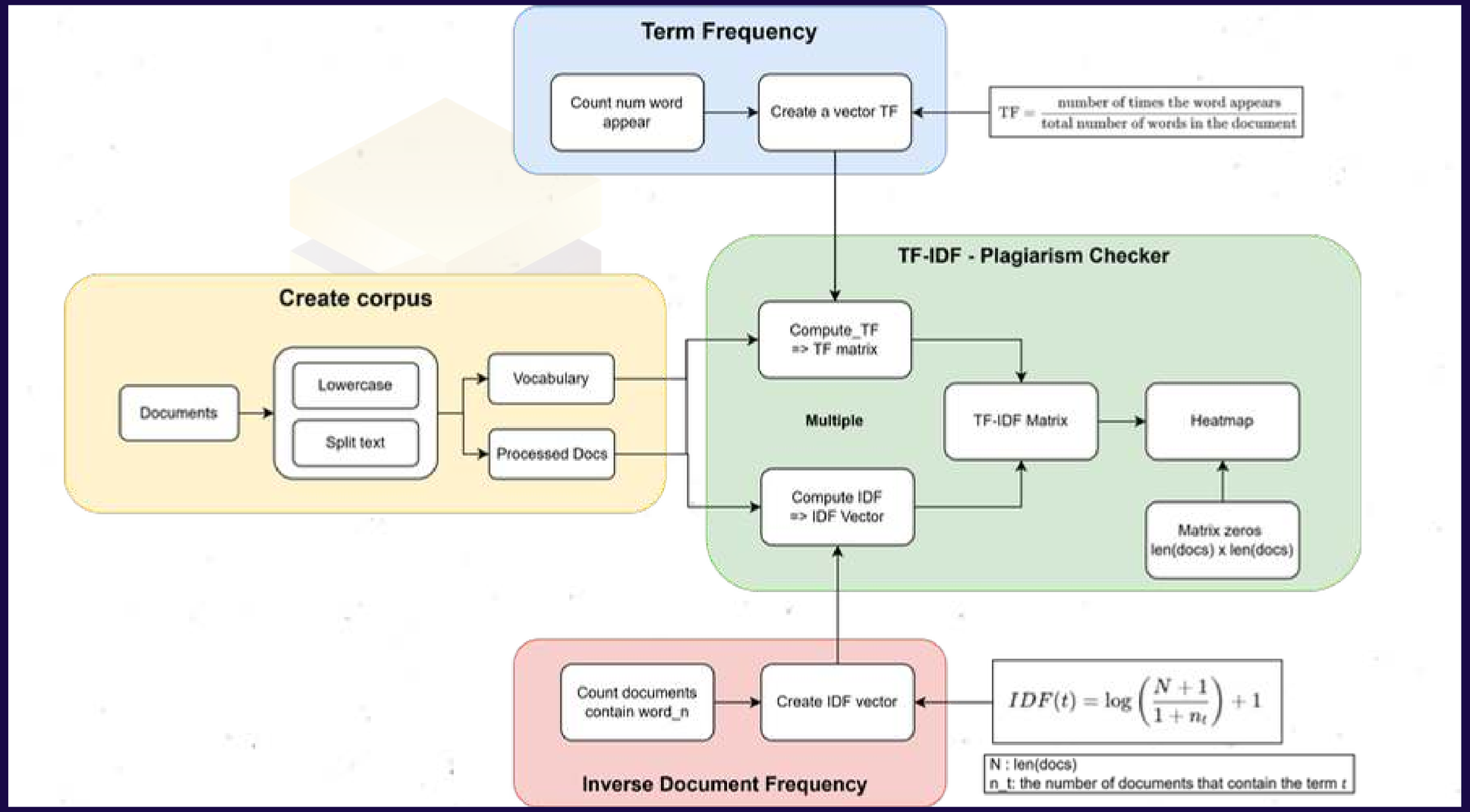


NLTK

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập trên lớp

PIPELINE



Các bước thực hiện

1. Cài đặt thư viện
2. Import thư viện
3. Create Corpus
4. Term Frequency
5. Inverse Document Frequency
6. TF-IDF
7. Cosine Similarity
8. Ứng dụng Correlation trong việc check đạo văn

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH

TF-IDF - Plagiarism Checker

1. Cài đặt thư viện

```
!pip install numpy matplotlib pandas seaborn
```



TF-IDF – Plagiarism Checker

2. Import datasets

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF – Plagiarism Checker

3. Create Corpus

```
def create_corpus(docs):
    """Tạo corpus từ danh sách văn bản"""
    processed_docs = [preprocess(doc) for doc in docs]
    vocabulary = sorted(set(word for doc in processed_docs for word in doc))
    return processed_docs, vocabulary

def preprocess(text):
    text = text.lower()  # Chuyển về chữ thường
    words = text.split()  # Tách từ
    return words
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF – Plagiarism Checker

4. Term Frequency

```
def compute_tf(word_list, vocabulary):
    """Tính TF (Term Frequency)"""
    word_count = {}
    for word in word_list:
        word_count[word] = word_count.get(word, 0) + 1
    tf_vector = np.array([word_count[word] / len(word_list) if word in word_count else 0 for word in vocabulary])
    return tf_vector
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF – Plagiarism Checker

5. Inverse Document Frequency

```
def compute_idf(docs, vocabulary):
    """Tính IDF (Inverse Document Frequency)"""
    num_docs = len(docs)
    idf_vector = np.array([
        np.log((num_docs + 1) / (1 + sum(1 for doc in docs if word in doc))) + 1
        for word in vocabulary
    ])
    return idf_vector
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF - Plagiarism Checker

6. TF-IDF

```python
def compute_tf_idf(docs):
    """Tính ma trận TF-IDF"""
    # Tạo corpus
    processed_docs, vocabulary = create_corpus(docs)

    # Tính TF và IDF
    tf_matrix = np.array([compute_tf(doc, vocabulary) for doc in processed_docs])
    idf_vector = compute_idf(processed_docs, vocabulary)

    # Tính TF-IDF
    tf_idf_matrix = tf_matrix * idf_vector  # Nhân từng phần tử theo chiều dọc
    return tf_idf_matrix
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF – Plagiarism Checker

7. Cosine Similarity

```python
def cosine_similarity(vec1, vec2):
    """Tính độ tương đồng cosine giữa hai vector"""
    dot_product = np.dot(vec1, vec2)
    norm1 = np.linalg.norm(vec1)
    norm2 = np.linalg.norm(vec2)
    return dot_product / (norm1 * norm2) if norm1 * norm2 != 0 else 0
```

ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF – Plagiarism Checker

8. Ứng dụng Correlation trong việc check đạo văn

```python
def check_plagiarism_all(docs):
    """Tính độ tương đồng giữa tất cả các cặp văn bản"""
    tf_idf_matrix = compute_tf_idf(docs)
    num_docs = len(docs)

    similarity_matrix = np.zeros((num_docs, num_docs))

    for i in range(num_docs):
        for j in range(num_docs):
            similarity_matrix[i, j] = cosine_similarity(tf_idf_matrix[i], tf_idf_matrix[j])

    return similarity_matrix
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF – Plagiarism Checker

8. Main

```
docs = [
  "Học máy là một nhánh của trí tuệ nhân tạo",
  "Trí tuệ nhân tạo bao gồm học máy và mạng nơ ron",
  "Mạng nơ ron là một mô hình quan trọng trong học sâu",
  "Học sâu là một lĩnh vực của trí tuệ nhân tạo và học máy"
]

similarity_matrix = check_plagiarism_all(docs)

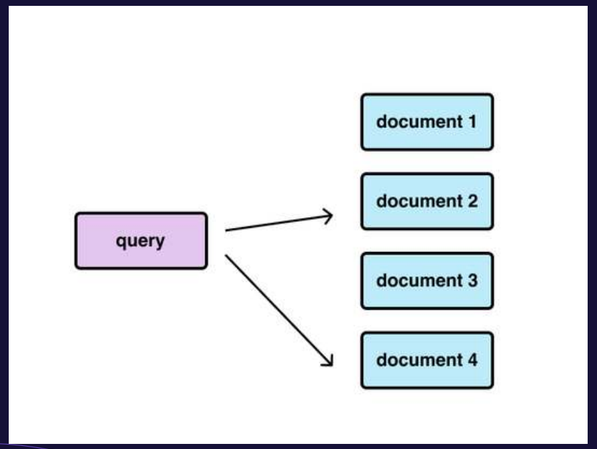
plt.figure(figsize=(10, 8))
sns.heatmap(similarity_matrix , annot =True , fmt=".3f", cmap ="YlOrRd")
plt.show()
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



Bài tập thực hành

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



TF-IDF - Text Retrieval

Bài tập thực hành

CÔNG NGHỆ & FRAMEWORK



NumPy

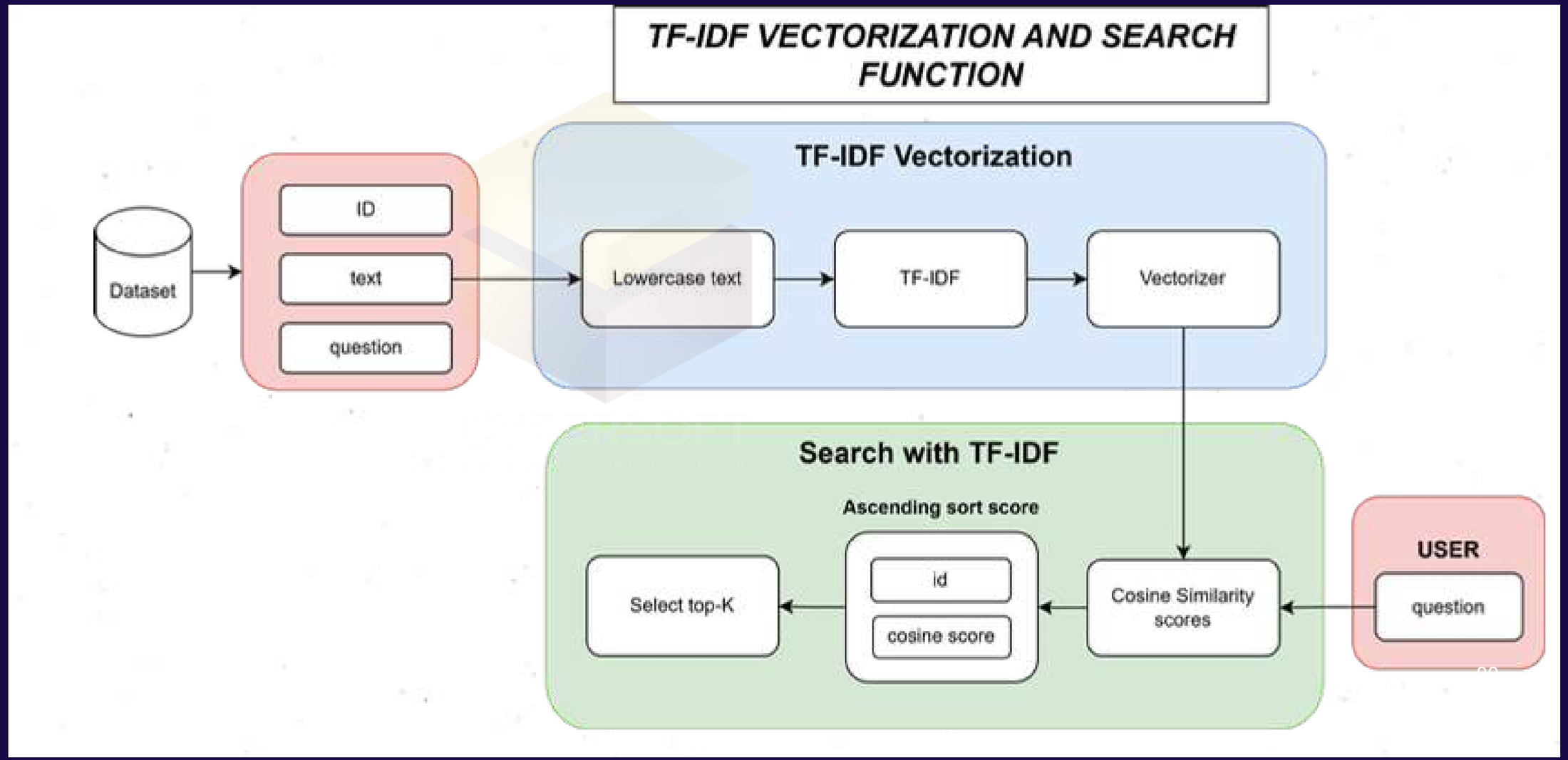
spaCy

NLTK

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành

PIPELINE



Các bước thực hiện

1. Cài đặt thư viện
2. Import thư viện
3. Chuẩn bị dataset
4. Tìm kiếm dựa trên TF-IDF
5. Tìm kiếm dựa trên độ tương đồng (Correlation)

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF – Text Retrieval

1. Cài đặt thư viện

```
!pip install numpy matplotlib pandas scikit-learn
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF - Text Retrieval

2. Import datasets

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF - Text Retrieval

3. Chuẩn bị dataset

```
vi_data_df = pd.read_csv("./vi_text_retrieval.csv")
context = vi_data_df['text']
context = [doc.lower() for doc in context]

tfidf_vectorizer = TfidfVectorizer()
context_embedded = tfidf_vectorizer.fit_transform(context).toarray()
```

```
context_embedded[7][0]
```

```
np.float64(0.31126580760710637)
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

TF-IDF - Text Retrieval

4. Tìm kiếm dựa trên TF-IDF

```python
def tfidf_search(question, tfidf_vectorizer, top_d=5):
    # Lowercasing before encoding
    query_embedded = tfidf_vectorizer.transform([question.lower()]).toarray()[0]
    cosine_scores = cosine_similarity([query_embedded], tfidf_vectorizer.transform(context).toarray())[0]

    # Get top k cosine score and index its
    results = []
    for idx in cosine_scores.argsort()[-top_d:][::-1]:
        doc_score = {
            'id': idx,
            'cosine_score': cosine_scores[idx]
        }
        results.append(doc_score)
    return results

question = vi_data_df.iloc[0]['question']
results = tfidf_search(question, tfidf_vectorizer, top_d=5)
results
```

```
[{'id': np.int64(0), 'cosine_score': np.float64(0.6279910475266973)},
 {'id': np.int64(97), 'cosine_score': np.float64(0.2114579586225163)},
 {'id': np.int64(136), 'cosine_score': np.float64(0.17714753149794468)},
 {'id': np.int64(384), 'cosine_score': np.float64(0.1597121839588734)},
 {'id': np.int64(118), 'cosine_score': np.float64(0.1578067474650028)}]
```

TF-IDF – Text Retrieval

5. Tìm kiếm dựa trên độ tương đồng (Correlation)

```python
def corr_search(question, tfidf_vectorizer, top_d=5):
    # Lowercasing before encoding
    query_embedded = tfidf_vectorizer.transform([question.lower()]).toarray()[0]
    cosine_scores = cosine_similarity([query_embedded], tfidf_vectorizer.transform(context).toarray())[0]

    # Get top k cosine score and index its
    results = []
    for idx in cosine_scores.argsort()[-top_d:][::-1]:
        doc_score = {
            'id': idx,
            'corr_score': cosine_scores[idx]
        }
        results.append(doc_score)
    return results

question = vi_data_df.iloc[0]['question']
results = corr_search(question, tfidf_vectorizer, top_d=5)
results
```

```python
question = vi_data_df.iloc[5]['question']
results = corr_search(question, tfidf_vectorizer, top_d=5)
results
```

CYBERSOFT
LẬP TRÌNH & CÔNG NGHỆ